Unit 5: Stacks and Queues

CPTR 242 — Sequential and Parallel Data Structures and Algorithms

Walla Walla University

Part 1: The Stack ADT

Sections 5.1 – 5.3

🤔 You're editing a document and press Ctrl+Z three times.

Each undo reverses the most recent change — not the oldest one.

What ordering principle does this follow? What data structure naturally models it?

5.1 The Stack ADT

A **stack** is a collection that enforces **Last-In, First-Out (LIFO)** order.

The last item added is always the first item removed — like a stack of plates in a cafeteria.

Operation	Description	Complexity
`push(item)`	Add item to the top	O(1)
`pop()`	Remove and return the top item	O(1)
`peek()`	Return the top item without removing	O(1)
`isEmpty()`	True if the stack has no items	O(1)
`size()`	Return the number of items	O(1)

Every operation touches only the **top** — nothing else in the stack is accessed directly.

Real-World Stacks

You interact with stacks constantly:

- **Browser back button** — each visited page is pushed; back pops the most recent
- **Undo/redo** — every edit is pushed; Ctrl+Z pops; Ctrl+Y pushes to a redo stack
- **Function call stack** — every function call pushes a frame; return pops it
- **Expression evaluation** — compilers use stacks to evaluate `3 * (4 + 2)`
- **Matching brackets** — parsers use stacks to verify `({[]})` is balanced

The unifying theme: **you always need the most recently added item first**.

🤔 A stack built from a linked list — which end should be the "top"?

You have a singly-linked list: `head → [A] → [B] → [C] → null`

`push` and `pop` must both be O(1). Inserting and removing at the front is O(1). Inserting and removing at the back requires walking the whole list — O(n).

So which end of the linked list should the top of the stack be?

5.2 Stacks Using Linked Lists

The **head of the linked list is the top of the stack**.

```
push(10):   head → [10] → null
push(20):   head → [20] → [10] → null
push(30):   head → [30] → [20] → [10] → null
pop():      returns 30,  head → [20] → [10] → null
peek():     returns 20,  list unchanged
```

Every push inserts at the front. Every pop removes from the front. Both are O(1) — no traversal, no resizing.

> The linked list gives the stack **unbounded size** — it grows as large as the heap allows. No capacity planning required.

5.3 C++: Array-Based Stack

Arrays give a simpler implementation with better cache performance — at the cost of a fixed maximum size.

```
class Stack {
    int  data[1000];   // fixed capacity
    int  top = -1;     // index of top element (-1 = empty)
public:
    void push(int val) {
        if (top == 999) throw overflow_error("Stack full");
        data[++top] = val;
    }

    int pop() {
        if (top == -1) throw underflow_error("Stack empty");
        return data[top--];
    }

    int peek() const {
        if (top == -1) throw underflow_error("Stack empty");
        return data[top];
    }
```

Array Stack vs. Linked Stack

Property	Array-based	Linked list
Memory	O(n) — pre-allocated block	O(n) — one node per element
Max size	Fixed at compile/construction time	Bounded only by heap
Cache performance	Excellent — contiguous	Poor — scattered nodes
Push / Pop	O(1) — index arithmetic	O(1) — pointer swap
Overflow possible?	Yes — if capacity exceeded	No — heap permitting
Overhead per element	None	One pointer per node

🤔 Use a stack to check if brackets are balanced.

Given the string `"({[]})"` — how would you use a stack to verify every open bracket has a matching close bracket in the right order?

Walk through it. What do you push? When do you pop? When do you know it's valid or invalid?

Stack Application: Bracket Matching

```
bool isBalanced(const string& s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '{' || c == '[')
            st.push(c);                          // push every opener
        else if (c == ')' || c == '}' || c == ']') {
            if (st.empty()) return false;        // closer with nothing open
            char top = st.top(); st.pop();
            if ((c == ')' && top != '(') ||      // mismatched pair
                (c == '}' && top != '{') ||
                (c == ']' && top != '[')) return false;
        }
    }
    return st.empty();                           // valid iff all openers closed
}
```

Part 2: The Queue ADT

Sections 5.4 – 5.6

🤔 A printer receives jobs from 10 people simultaneously.

Everyone submits at roughly the same time. How should the printer decide which job to print first?

What ordering principle is fair here? How is it different from the stack?

5.4 The Queue ADT

A **queue** is a collection that enforces **First-In, First-Out (FIFO)** order.

The first item added is the first item removed — like a line at a coffee shop.

Operation	Description	Complexity
`enqueue(item)`	Add item to the **back**	O(1)
`dequeue()`	Remove and return from the **front**	O(1)
`peek() / front()`	Return front item without removing	O(1)
`isEmpty()`	True if no items	O(1)
`size()`	Return number of items	O(1)

Unlike a stack, a queue has **two active ends**: items enter at the back and leave from the front.

Real-World Queues

- **Print queues** — jobs print in submission order
- **OS process scheduling** — CPU time allocated in arrival order
- **Network packet buffers** — routers forward packets FIFO
- **BFS (Breadth-First Search)** — graph traversal explores level by level using a queue
- **Keyboard input buffer** — keystrokes are processed in the order typed

The unifying theme: **fairness and order preservation**. The thing that waited longest goes next.

🤔 A queue built from a linked list needs two O(1) ends.

`enqueue` adds to the back. `dequeue` removes from the front.

A singly-linked list with only a `head` pointer can remove from the front in O(1) — but adding to the back requires walking the whole list.

What single addition to the linked list data structure fixes this?

5.5 Queues Using Linked Lists

Add a `tail` **pointer** so both ends are O(1):

- `enqueue` → append to `tail` — O(1)
- `dequeue` → remove from `head` — O(1)

> When the last element is dequeued, both `head` and `tail` must be set to `null`. Missing this is the most common bug in linked-list queues.

🤔 Can you implement a queue using a plain array?

Imagine `front` and `back` as indices into the array.

`dequeue` increments `front`. `enqueue` increments `back`.

What happens after many enqueue/dequeue cycles?
Does the array "fill up" even if it's logically empty?

5.6 C++: Array-Based Queue

A naive array wastes space — `front` drifts right leaving unused slots. The fix is a **circular array**.

```
class Queue {
    int  data[1000];
    int  front = 0, back = 0, count = 0;
    static const int CAP = 1000;
public:
    void enqueue(int val) {
        if (count == CAP) throw overflow_error("Queue full");
        data[back] = val;
        back = (back + 1) % CAP;   // wrap around
        count++;
    }
    int dequeue() {
        if (count == 0) throw underflow_error("Queue empty");
        int val = data[front];
        front = (front + 1) % CAP; // wrap around
        count--;
```

The Circular Array Trick

The modulo operator `% CAP` makes the array behave like a ring:

```
Indices:  0   1   2   3   4   5   6   7  (CAP = 8)

After enqueue A, B, C:
          [A] [B] [C] [ ] [ ] [ ] [ ] [ ]
           ↑                 ↑
         front             back

After dequeue (removes A):
          [ ] [B] [C] [ ] [ ] [ ] [ ] [ ]
               ↑            ↑
             front        back

After enqueue D, E, F, G, H, I (wraps around):
          [I] [B] [C] [D] [E] [F] [G] [H]
               ↑   ↑
             front back (full!)
```

Part 3: Linked Implementations Side by Side

Section 5.7

🤔 How much code does the stack share with the queue?

Both use a linked list internally. Both have push/pop (or enqueue/dequeue) as O(1) pointer operations.

If you were writing both from scratch, what would be different? What would be nearly identical?

Stack vs. Queue: The One Key Difference

Both use the same `Node` struct. Both `push` / `enqueue` create a new node. Both `pop` / `dequeue` delete a node.

The difference is **where the active end is**:

	Stack	Queue
Insert at	front (head)	back (tail)
Remove from	front (head)	front (head)
Extra pointer needed?	No — head only	Yes — tail required
LIFO or FIFO?	LIFO	FIFO

The queue needs a `tail` pointer because its insert and remove ends are different. The stack's insert and remove are both at `head` — no second pointer needed.

Part 4: The Deque ADT

Section 5.8

🤔 What if you needed a data structure that was both a stack *and* a queue?

You want to add and remove from **either end** efficiently.

Can you think of an application where you'd want that flexibility? What would the operations be called?

5.8 The Deque ADT

A **deque** (double-ended queue, pronounced "deck") supports O(1) insert and remove at **both** ends.

Operation	Description
`pushFront(item)`	Add to the front
`pushBack(item)`	Add to the back
`popFront()`	Remove and return from the front
`popBack()`	Remove and return from the back
`peekFront()`	Return front item
`peekBack()`	Return back item
`isEmpty()`	True if empty

Real-World Deques

- **Browser history with forward button** — back = popFront, forward = pushFront, new visit = pushFront (discarding any forward history beyond current)
- **Sliding window algorithms** — e.g. finding the maximum in a moving window of the last k elements
- **Work-stealing schedulers** — a thread pushes and pops from the back of its own deque; idle threads steal from the front of another thread's deque
- **Palindrome check** — push all characters, then pop from both ends simultaneously and compare

> `std::deque` in C++ implements this ADT. Internally it uses a segmented array — not a linked list — to give O(1) amortized access at both ends while maintaining reasonable cache performance.

🤔 How would you implement a deque efficiently?

A singly-linked list can remove from the front in O(1) — but removing from the back requires walking the whole list.

Which list variant from Unit 4 supports O(1) operations at both ends? Why does it work where a singly-linked list fails?

Deque Implementation: Doubly-Linked List

A **doubly-linked list with head and tail pointers** gives O(1) at both ends.

🤔 `std::stack` and `std::queue` in C++ are not containers — they're *adaptors*.

They wrap another container (`std::deque` by default) and restrict its interface.

Why would you design a data structure by restricting a more powerful one rather than building it from scratch?

Why Adaptors?

- **Enforces the contract** — code using a `std::stack` cannot accidentally call `v[i]` or `insert(begin)`. The restricted interface communicates intent.
- **Documents design decisions** — a function that takes a `std::stack<int>` tells every reader: this is LIFO, that's the invariant.
- **Prevents misuse** — if you use a `std::vector` as a stack, nothing stops a colleague from calling `v.insert(v.begin(), x)` and silently breaking the LIFO assumption.
- **Swappable implementation** — change the underlying container without touching any calling code.

> This is the ADT principle in action: the interface constrains behavior, and the implementation is an internal detail.

Project Proposals!

> or

Lab 4!